

Machine Learning

CSE 343/543

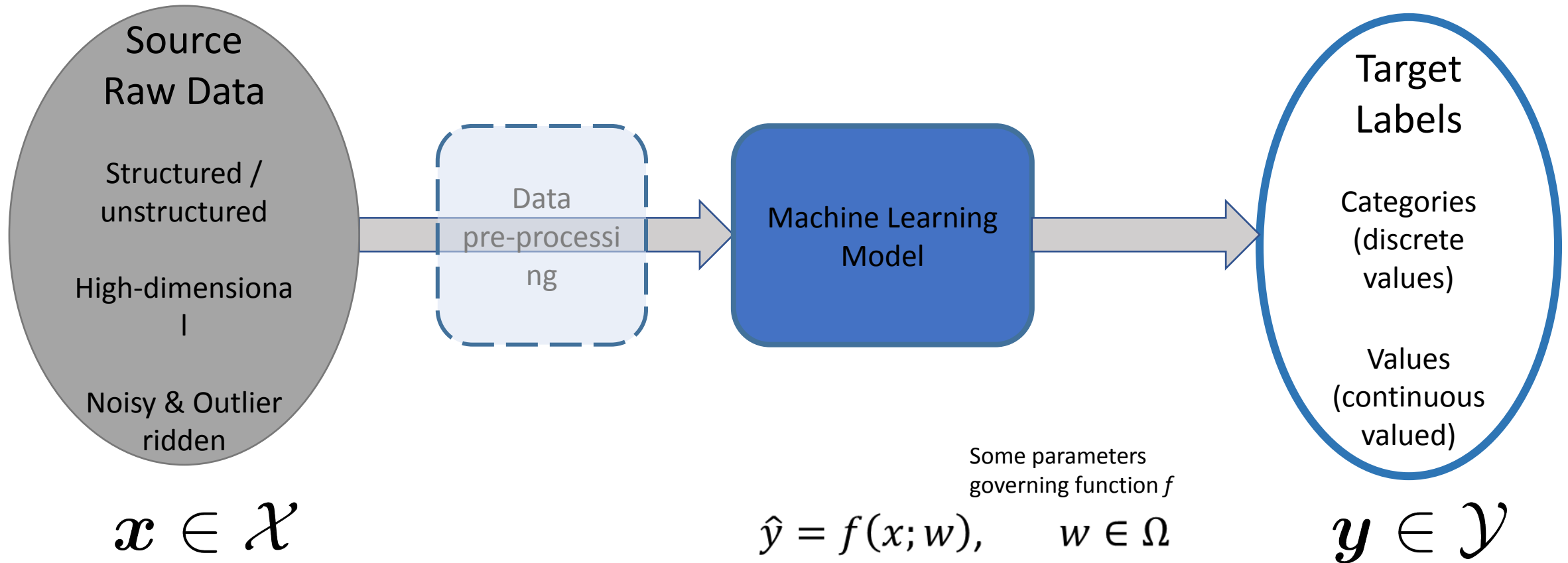
Lecture 2

Empirical Risk Minimization

Outline

- Learning Machines
- Risk Functional
- Calculus of variations
- Empirical Risk Minimization

Machine Learning as a Function Mapping



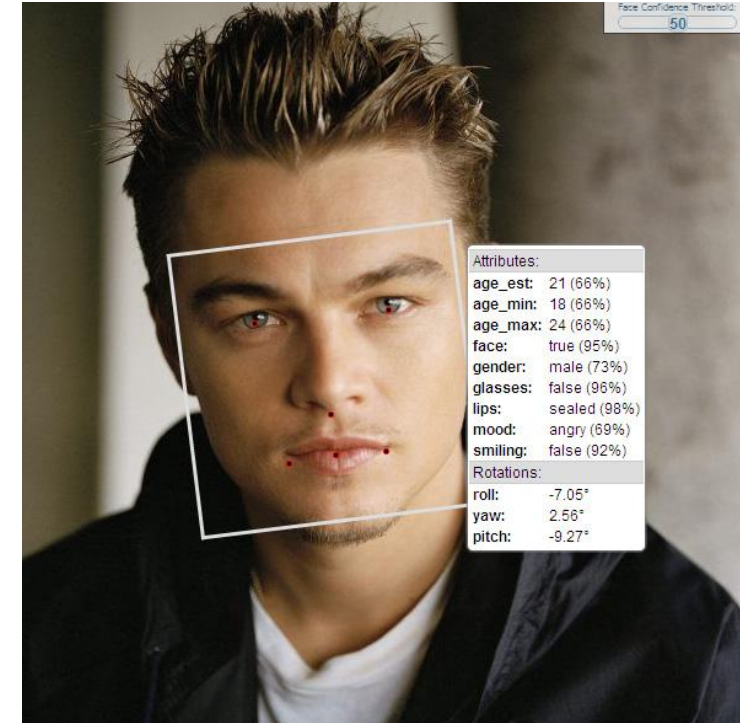
- Objective of supervised learning models
 - To predict the target labels on data not seen before (unseen data).

Regression: Abstraction

- What are Regressors and how are they modelled?
 - Essentially a function mapping
- Regression
 - Predict [height / weight / age / head pose] based on [pictures / video] of a person
 - Predict [heart rate / BP] based on [close-up facial video]

$$f(x, w): \mathbb{R}^n \rightarrow \mathbb{R}^m$$

$$\mathcal{X} = \mathbb{R}^n \quad \mathcal{Y} = \mathbb{R}^m$$



Face analysis and recognition

Classification: Abstraction

- What are Classifiers and how are they modelled?
 - Essentially a function mapping

- Binary Classification

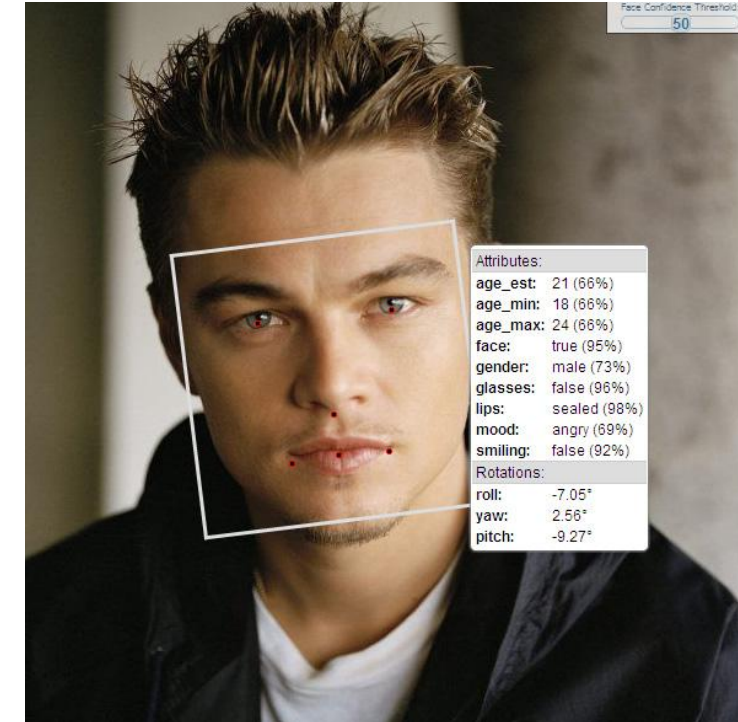
- Predict [gender / biological sex] based on [facial images / videos] of a person

$$f(x, w): \mathbb{R}^n \rightarrow \{-1, 1\}$$

- Multi-class Classification

- Animal species categorization (tiger, elephant, gaur, ...)
 - Road object detection (car, bike, bus, pedestrian, etc.)

$$f(x, w): \mathbb{R}^n \rightarrow \{0, 1, 2, \dots, k\}$$



Face analysis and recognition

$$\mathcal{X} = \mathbb{R}^n$$

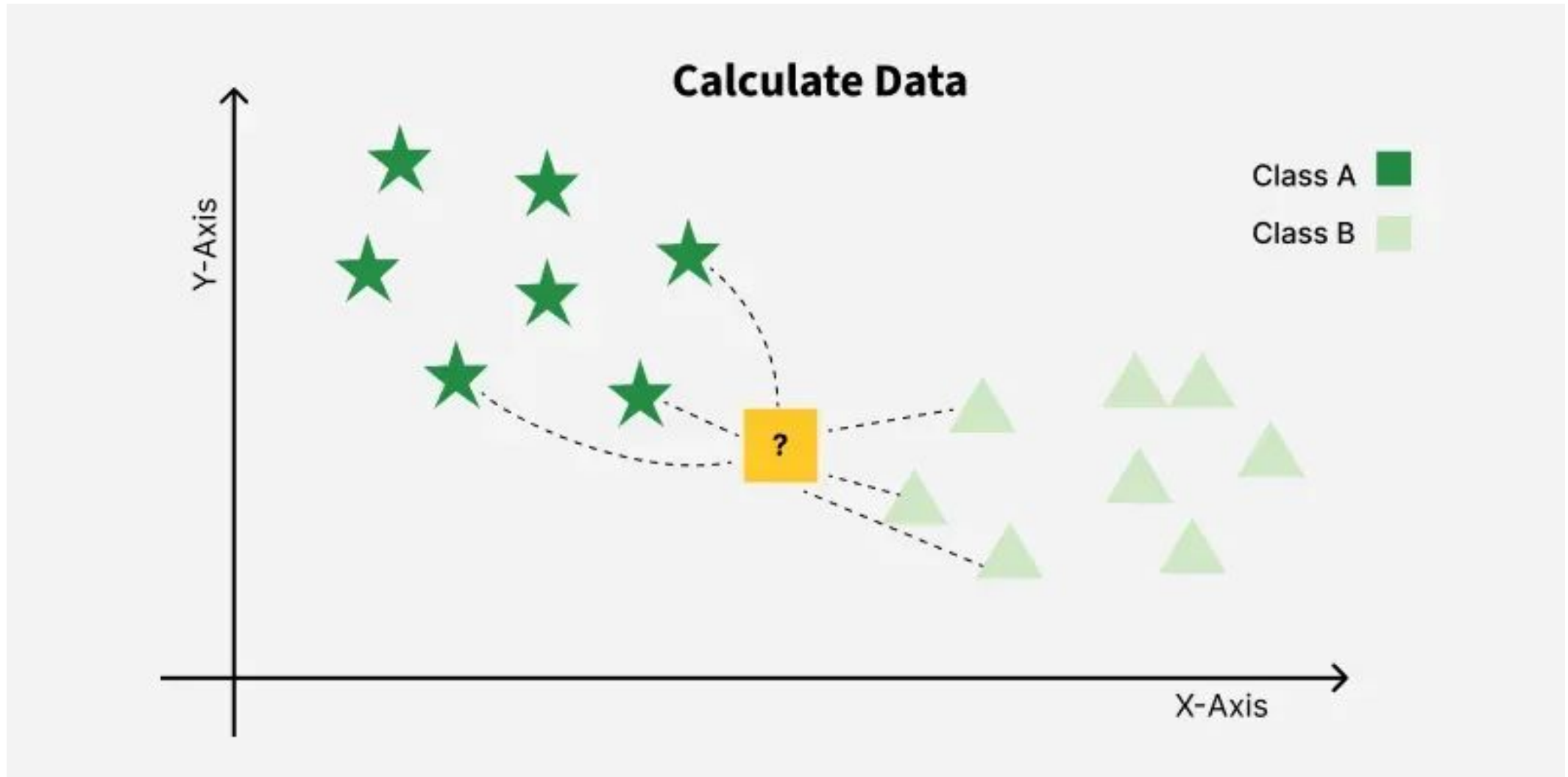
$$\mathcal{Y}_{bin} = \{0, 1\}$$

$$\mathcal{Y}_{multi} = \{0, 1, \dots, k\}$$

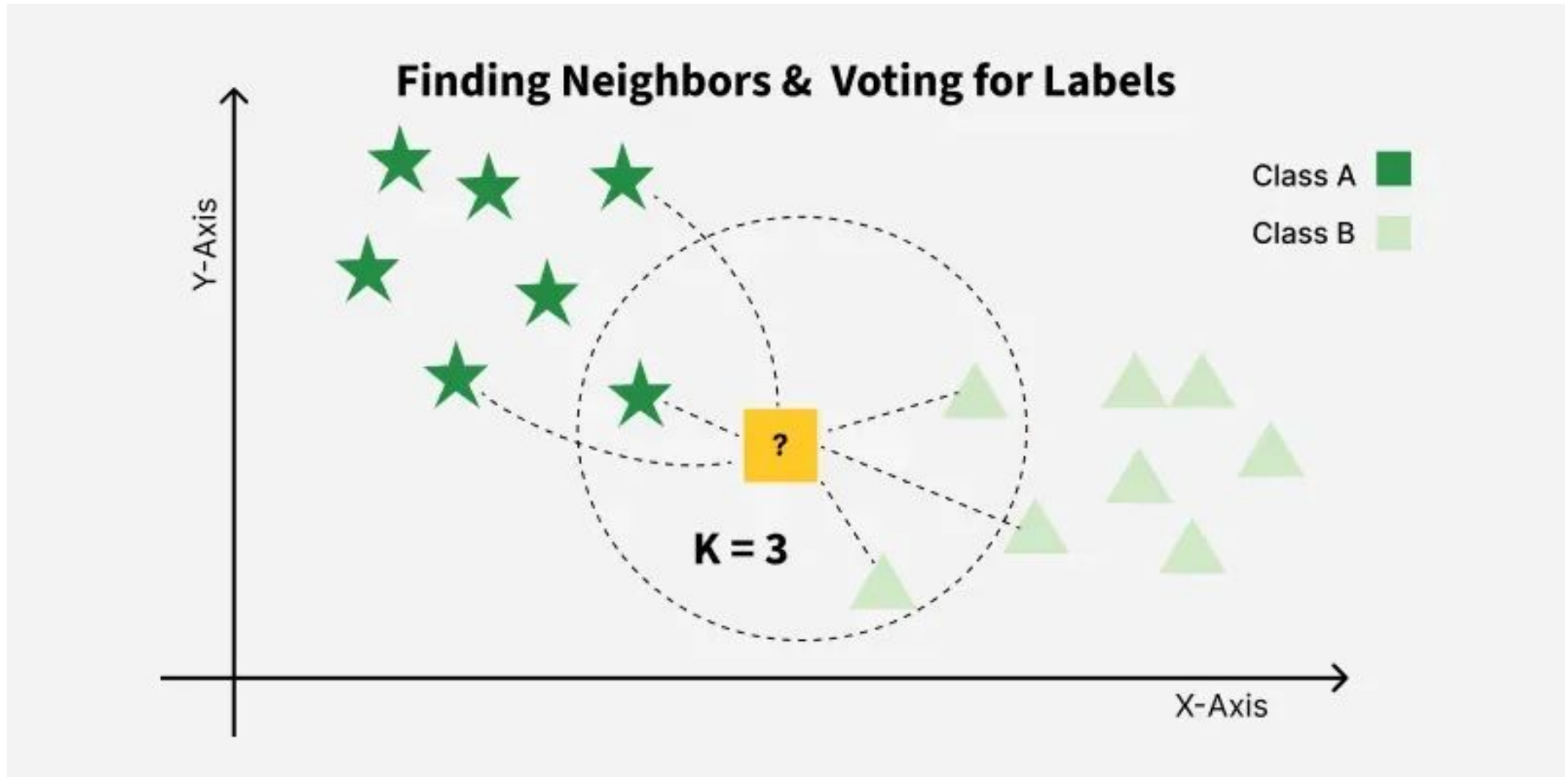
A Simple Classifier - K-Nearest Neighbor (KNN)



A Simple Classifier - K-Nearest Neighbor (KNN)

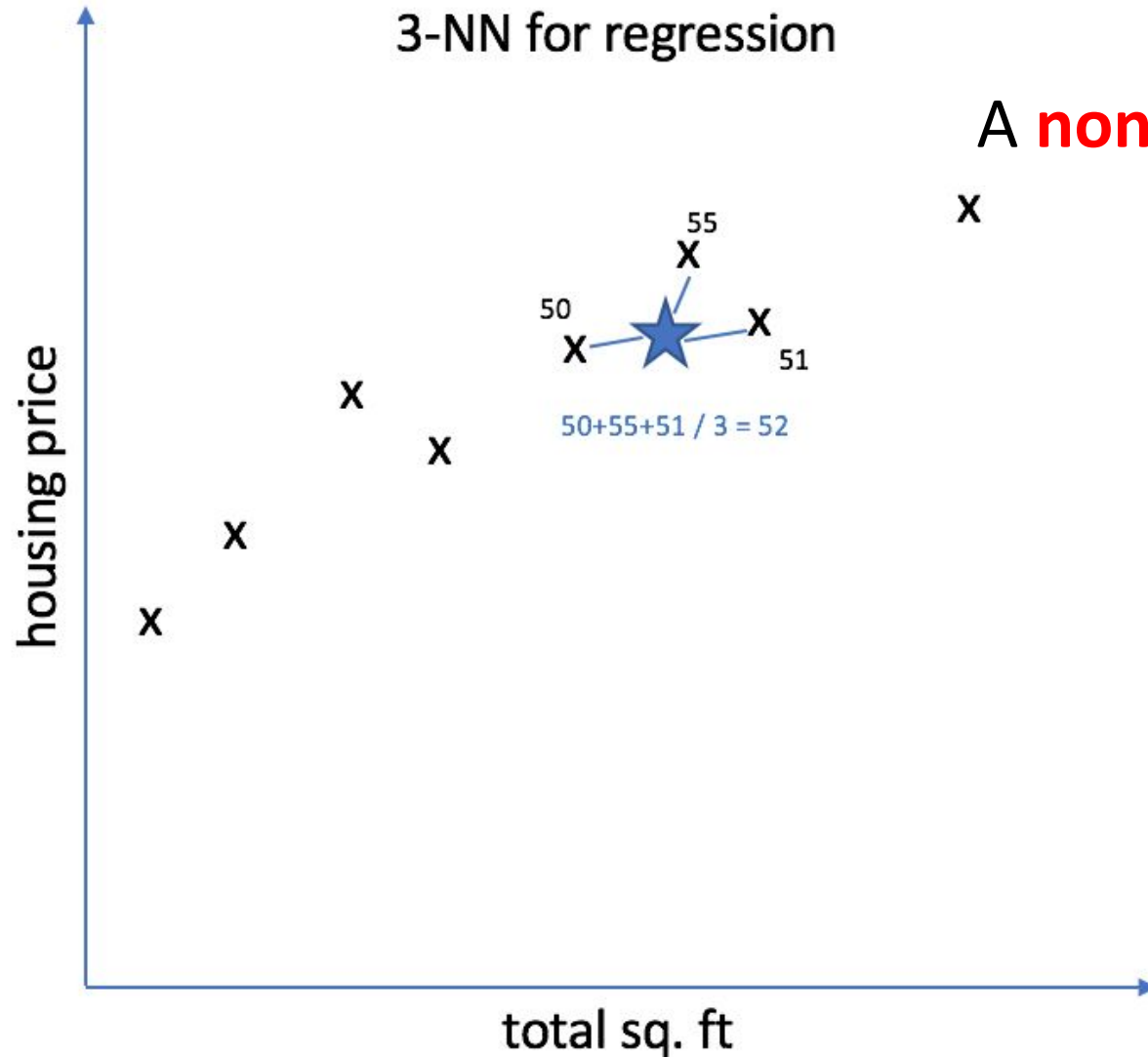


A Simple Classifier - K-Nearest Neighbor (KNN)



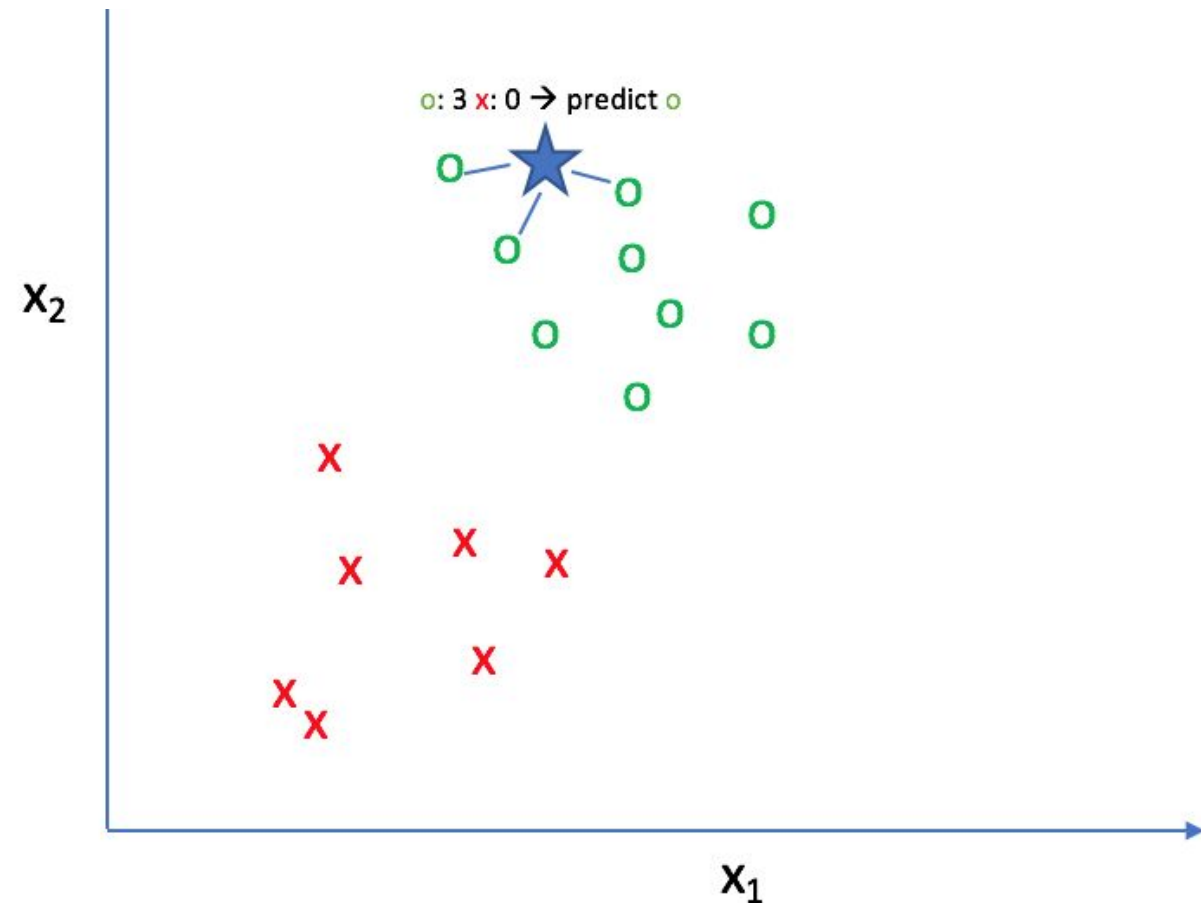
KNNs for Regression and Classification

3-NN for regression



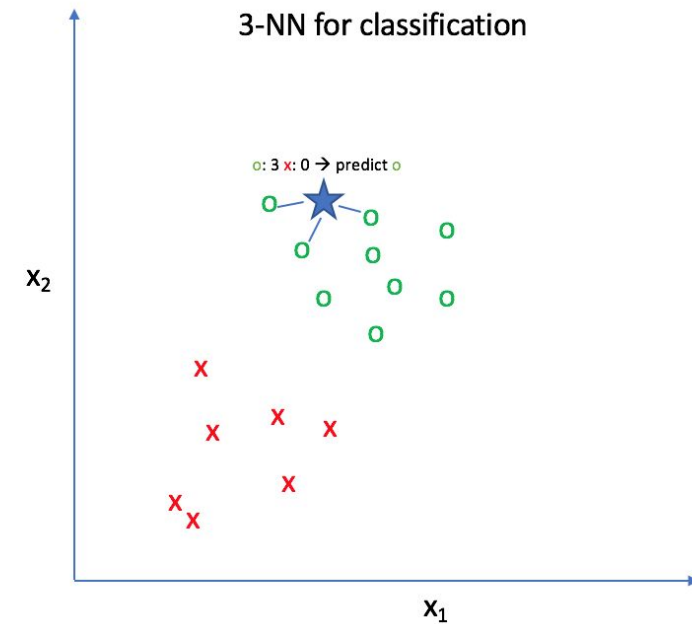
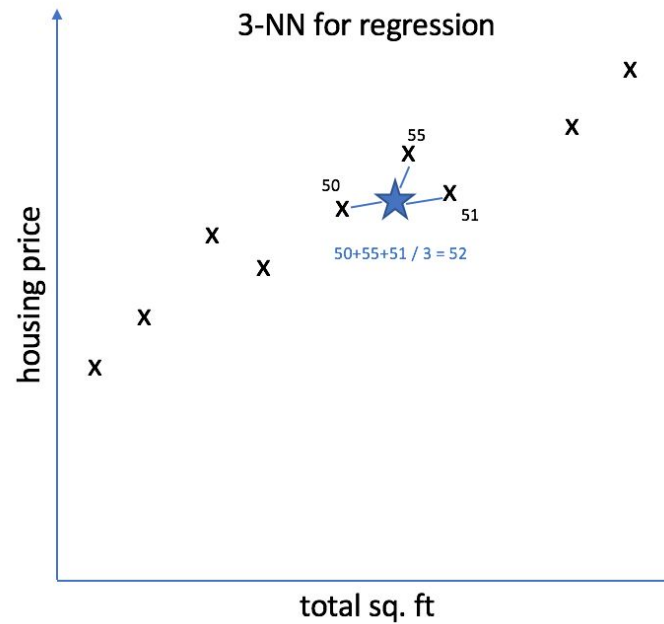
3-NN for classification

A **non-parametric** model

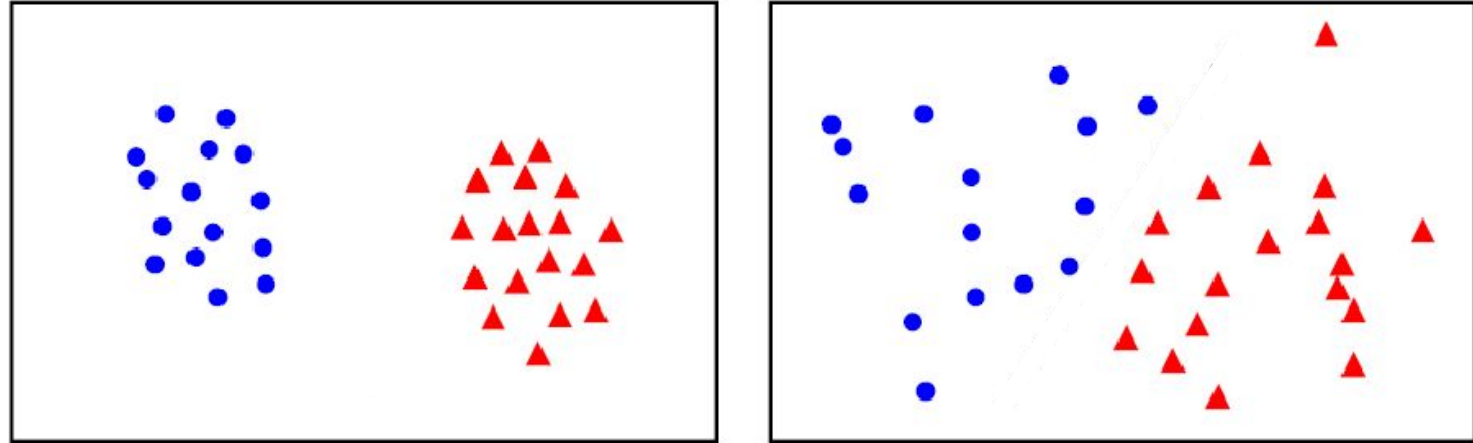


Problems with KNN?

- Distance / Similarity should be meaningful and must capture the notion of the 'class'
 - E.g., distance in pixel space may not reflect information about the face's ID.
- KNN is hard to scale
 - All data needs to be stored
 - Every inference needs k nearest neighbors computation



Alternatives to KNN type approach?

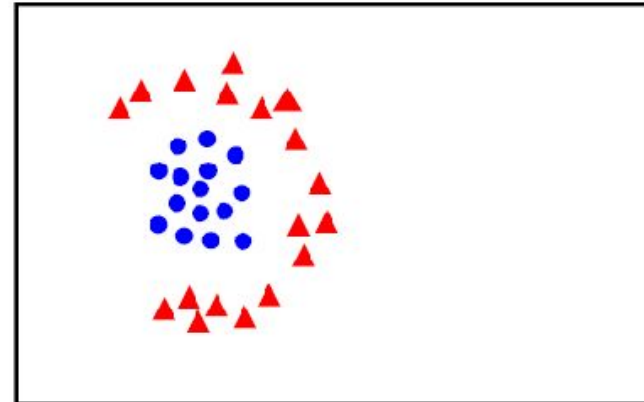
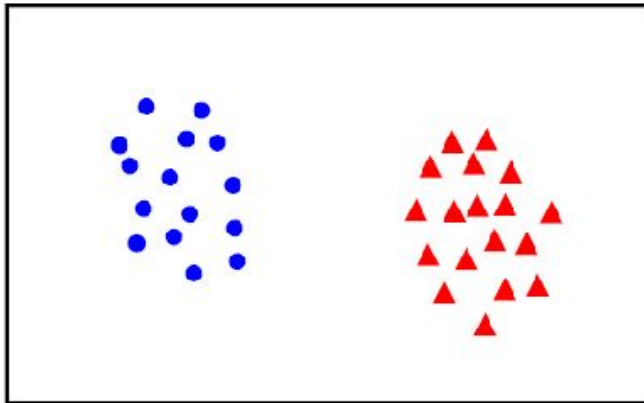


Binary Classification

Given training data $(\mathbf{x}_i, y_i)$ for $i = 1 \dots N$, with $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \{-1, 1\}$, learn a classifier $f(\mathbf{x})$ such that

$$f(\mathbf{x}_i) \begin{cases} \geq 0 & y_i = +1 \\ < 0 & y_i = -1 \end{cases}$$

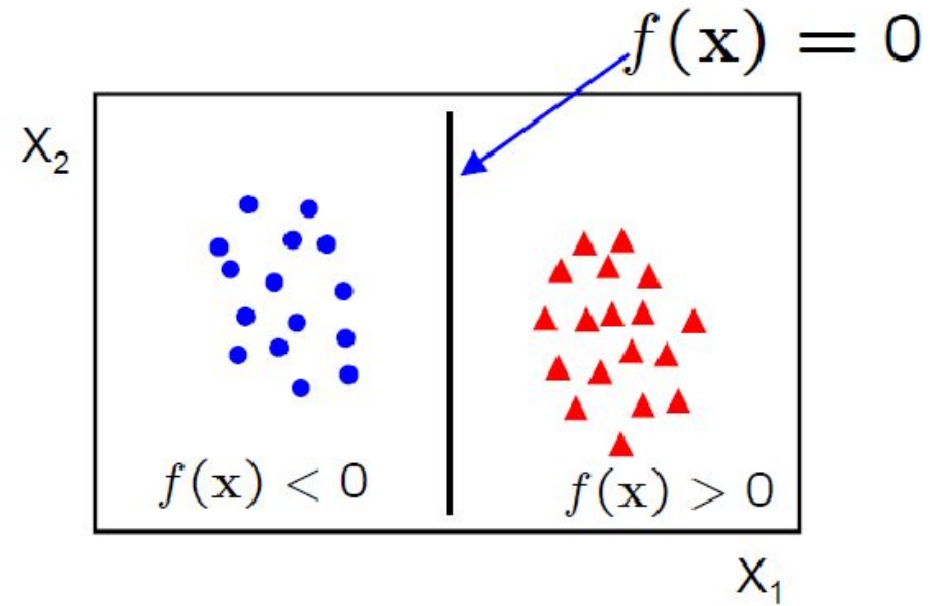
i.e. $y_i f(\mathbf{x}_i) > 0$ for a correct classification.



Linear classifiers

A linear classifier has the form

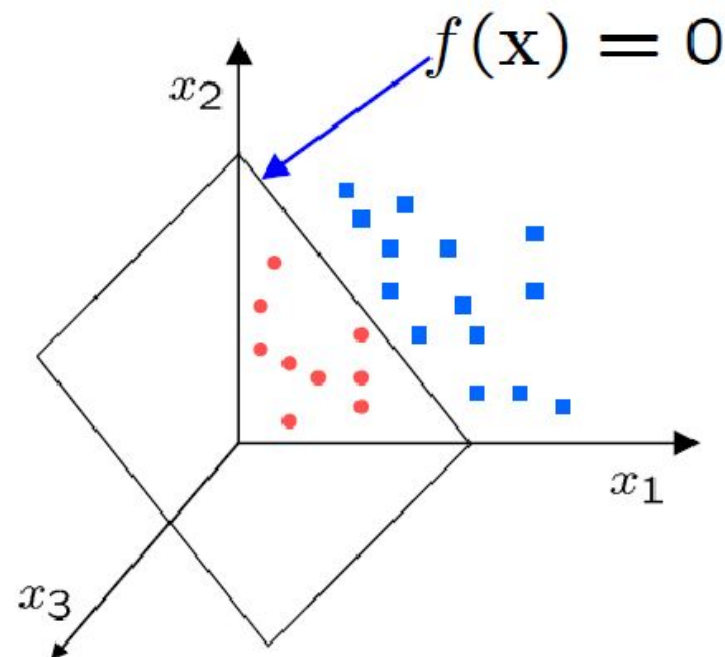
$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$



Linear classifiers

A linear classifier has the form

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$



- in 3D the discriminant is a plane, and in nD it is a hyperplane

For a K-NN classifier it was necessary to `carry' the training data

For a linear classifier, the training data is used to learn $\mathbf{w}$ and then discarded

Only $\mathbf{w}$ is needed for classifying new data

Learning Machines **mimic the supervisor!**

- Described through three components
 - **Raw Input Data:** random vectors x , i.i.d. from a fixed but unknown distribution $p(x)$
 - **Target Labels:** A supervisor (oracle) which returns an output vector y , for every input vector x , as per the conditional distribution $p(y|x)$, also fixed but unknown.
 - A learning machine capable of implementing a set of functions

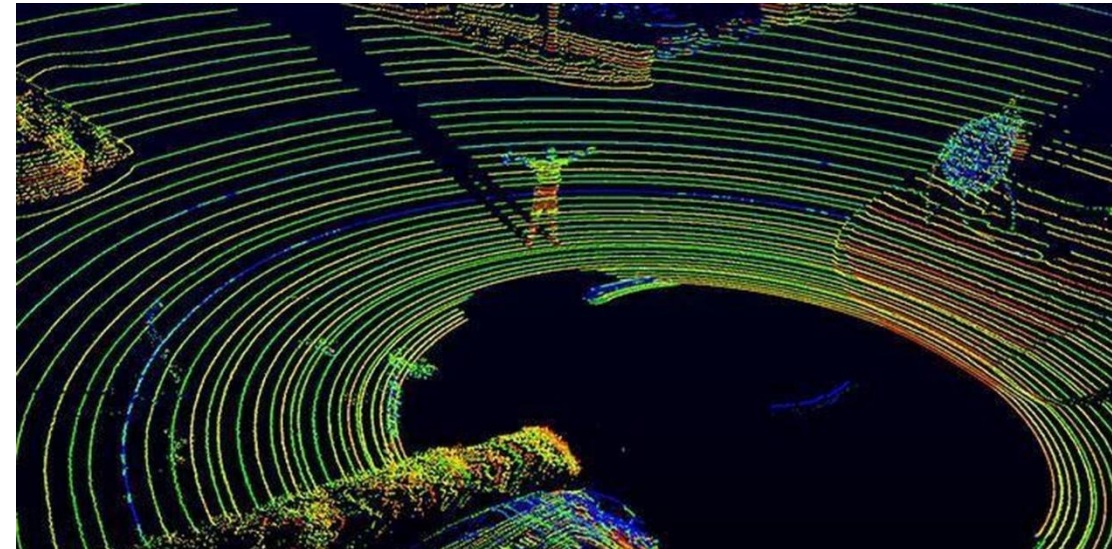
$$\hat{y} = f(x; w), \quad w \in \Omega$$

- The learning problem is to choose from the given set of functions the one which best approximates the supervisor's response.
 - The selection is based on 'training samples' (x_i, y_i) , $i = 1, \dots, N$

Regression: Concrete Example

LIDARs and Point Cloud

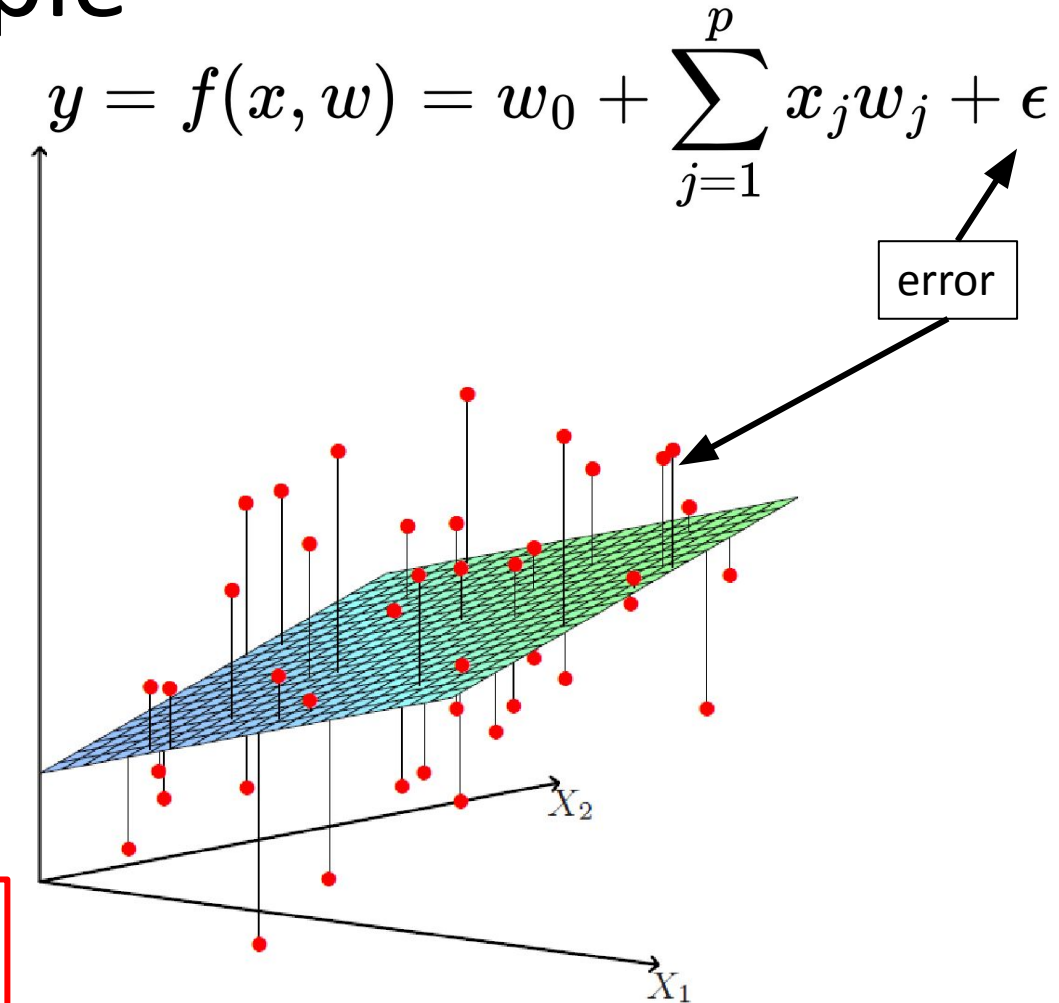
- Light Detection And Ranging (LiDaR)
 - Laser based range (distance) sensing
- Column of lasers sitting on a spinning motor
- Generates “Point Cloud” Data
- Problem: Find Road Surface
 - Assume planarity



Regression: Concrete Example

- Find Road Surface
 - Assume planarity
- Given the points, estimate parameters
 - Data/Feature
 - dimension (p=2) $x_i = (x_{i1}, x_{i2}, \dots, x_{ip})^T$
 - # training samples $(x_1, y_1) \dots (x_N, y_N)$
 - parameters $\mathbf{w} = (w_0, w_1, \dots, w_p)^T$
- Evaluation Metric
 - How good is the fitted plane?

$$\begin{aligned}\text{Total Error} &= \sum_{i=1}^N L(y_i, f(x_i, w)) = \sum_{i=1}^N (y_i - f(x_i, w))^2 \\ &= \sum_{i=1}^N \left(y_i - w_0 - \sum_{j=1}^p x_{ij} w_j \right)^2\end{aligned}$$



Very Important!!

Regression: Abstraction

- What are Regressors and how are they modelled?
 - Essentially a function mapping
- Regression
 - Predict [avg. enrolment in 2022, CGPA] based on [current enrolment, grade, job offer, package]
 - Predict [mutual fund value] based on [stock prices, inflation, PM's foreign visit expenses]

$$w \in \Omega$$

$$f(x, w) : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

Some parameters governing the function f .

Can be abstract parameters like:
Degree of polynomial +
coefficients

Loss functions

- To choose the best function, it makes sense to **minimize** a loss (or cost or discrepancy) between the **response of the supervisor and the learning machine**, given an input x

$$L(y, f(x, w))$$

- Since we want to minimize the loss over *all* samples, we are interested in minimizing the expected loss over all possible x and y

$$R(w) = \int \int p(x, y) L(y, f(x, w)) dx dy, \quad w \in \Omega$$

$p(x, y)$ is the joint distribution over data x , and target values y

Note: The risk (or expected loss) is defined over the entire population of x & y , as opposed to the samples (e.g., training data) alone.
The empirical risk (defined later) is an estimate of this risk based on the samples that are accessible

Loss Functions - Regression

p^{th} norm, $p \geq 1$

$$\|x\|_p = \left(\sum_{i=1}^d |x_i|^p \right)^{1/p}$$

- Least squares (L_2 -norm) minimization
 - y takes continuous values

$$L(y, f(x, w)) = \|y - f(x, w)\|_2^2$$

- L_1 -norm minimization

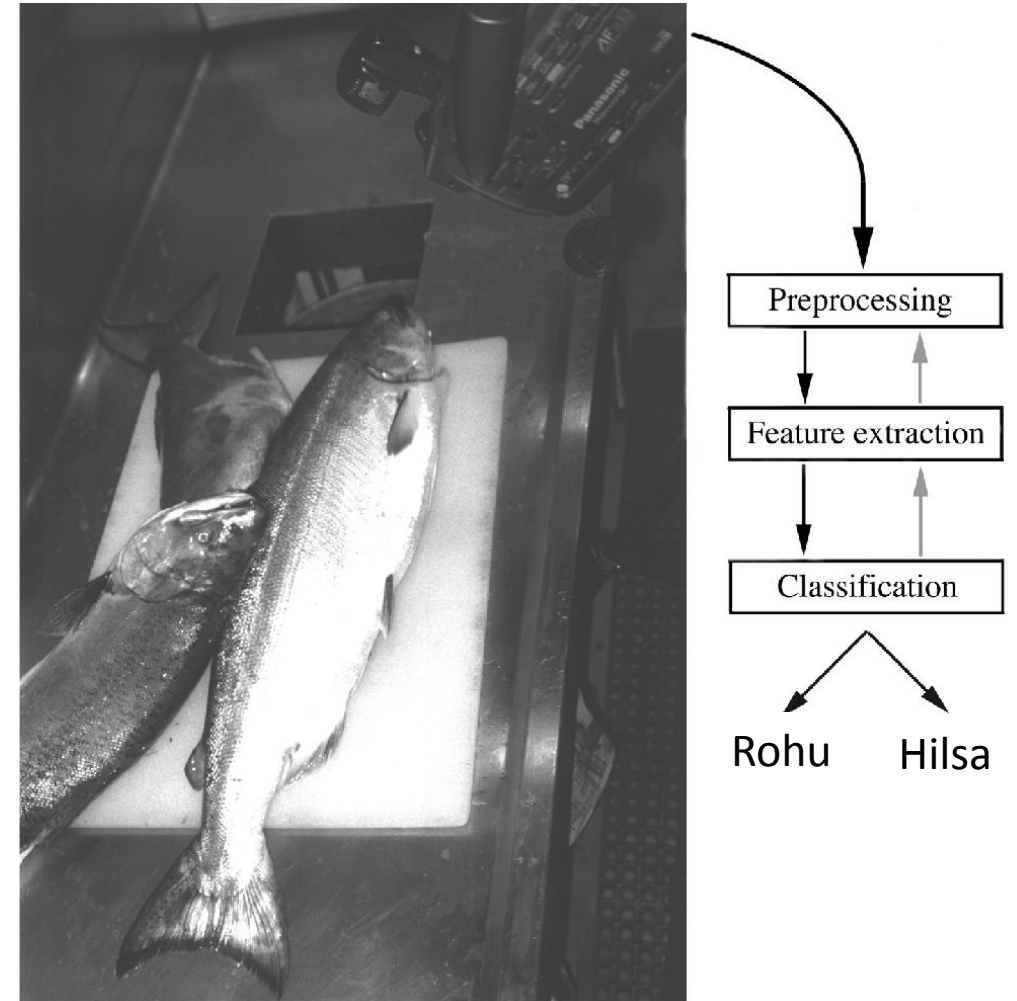
$$L(y, f(x, w)) = \|y - f(x, w)\|_1$$

- Other examples:
 - Huber loss (robust regression)

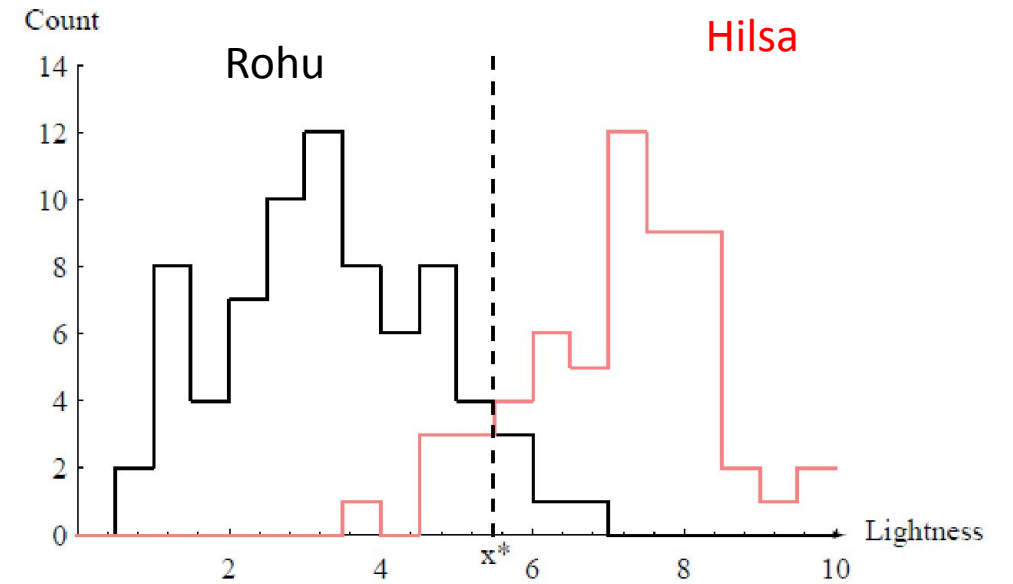
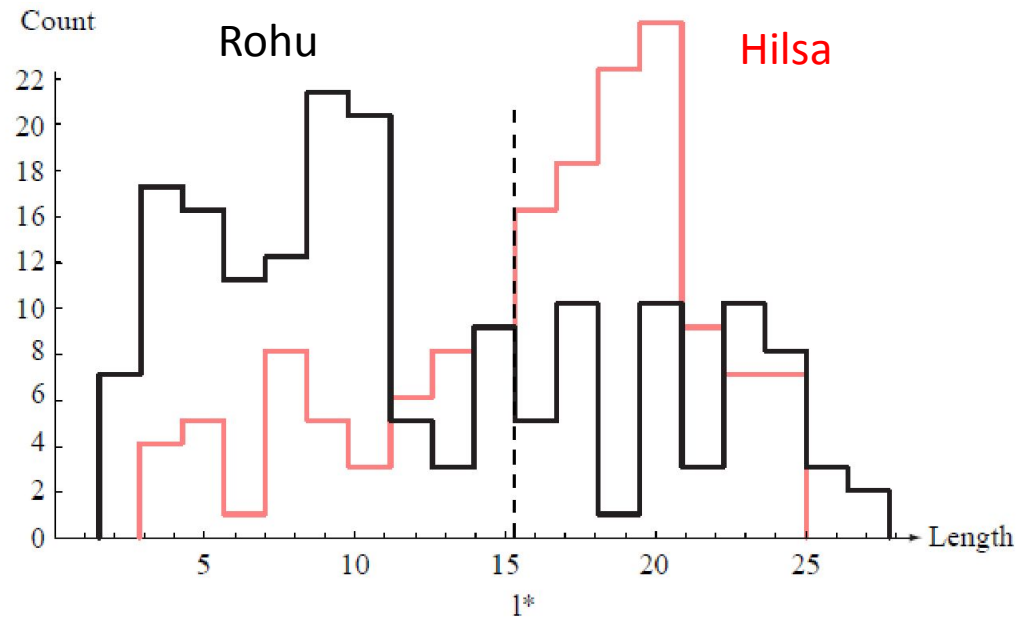
Classification: Concrete Example

Classification: Concrete Example

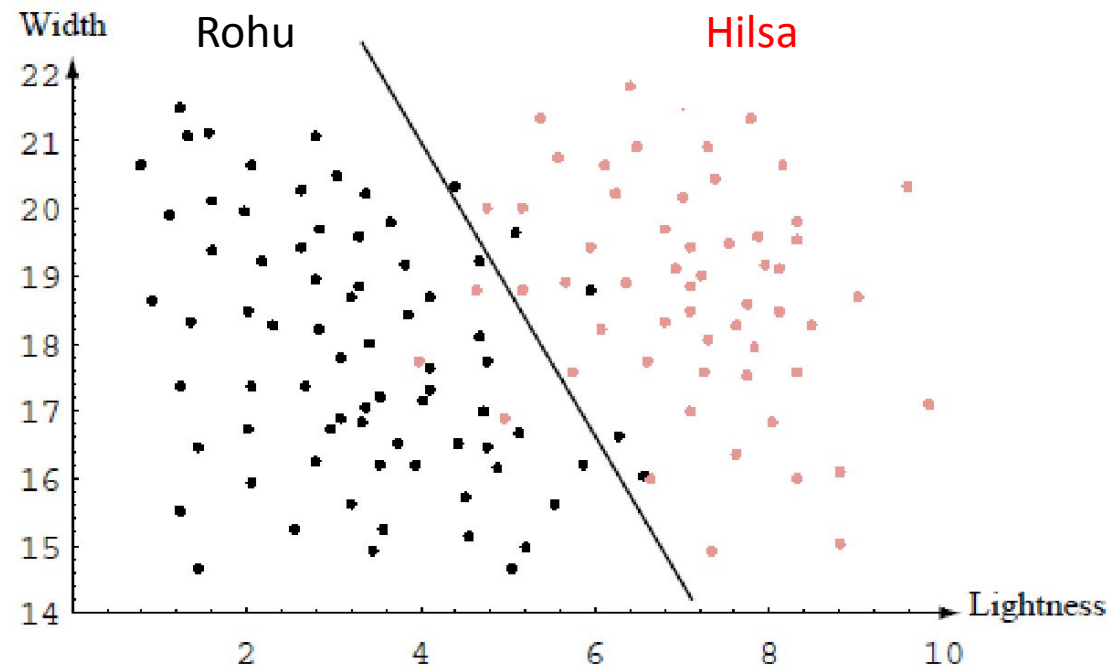
- Say you are 'Laloo', trying to make a good impression on 'Didi'
 - You need to know your fish, but you don't!
 - So you seek help from ML experts@IIITD to distinguish between 'Rohu' and 'Hilsa'
- 'Features' for classification
 - Length/width
 - brightness



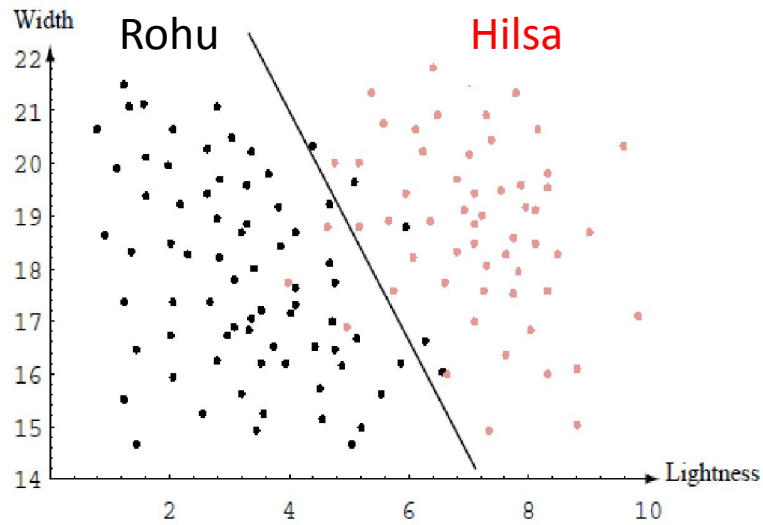
Classification: Rohu vs. Hilsa



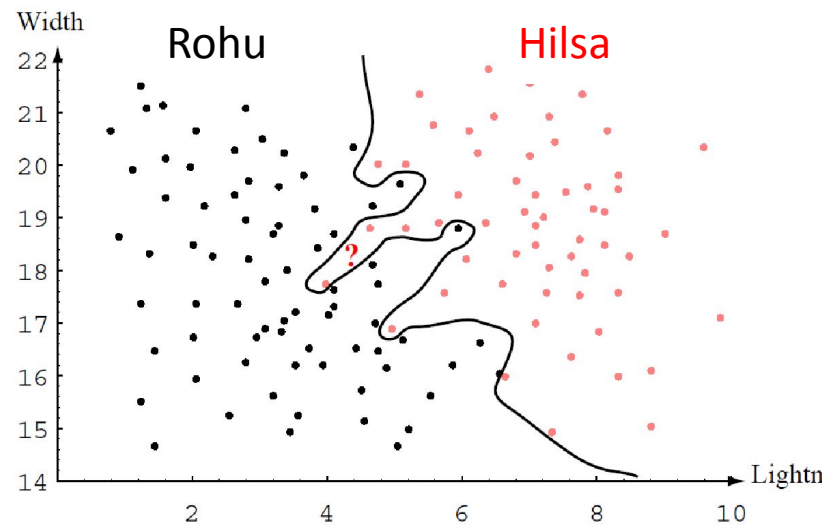
Classification: Rohu vs. Hilsa



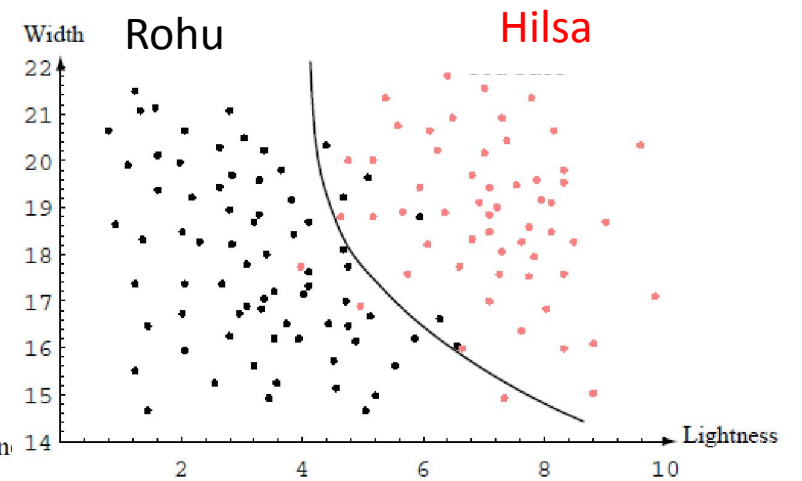
Model Complexity: A note



Too Simple?



Too Complex?



Just Right?

Loss Functions - Classification

- Binary Classification with equal weights on misclassification
 - Minimize a 0-1 loss

$$L(y, f(x, w)) = \begin{cases} 0 & \text{if } y = f(x, w) \\ 1 & \text{if } y \neq f(x, w) \end{cases}$$

- Classification with unequal weights on misclassification
 - Minimize a 0- 10^7 -500 loss

$$L(y, f(x, w)) = \begin{cases} 0 & \text{if } y = f(x, w) \\ 10^7 & \text{if } y = 1, f(x, w) = 0 \\ 500 & \text{if } y = 0, f(x, w) = 1 \end{cases}$$

Classification: Abstraction

- What are Classifiers and how are they modelled?
 - Essentially a function mapping

- Binary Classification

- Detection (Spam/no Spam; bomb/no bomb)

$$f(x, w) : \mathbb{R}^n \rightarrow \{-1, 1\}$$

- Multi-class Classification

- ADAS¹ (pedestrians, vehicles, barricades,...)
 - Biometric system (Saket, Akshit, Depanshu,...)

$$f(x, w) : \mathbb{R}^n \rightarrow \{0, 1, 2, \dots, k\}$$

$$w \in \Omega$$

Some parameters governing the function f .

Can be abstract parameters like:
one or several thresholds
one or several boundaries
(linear/nonlinear)

No. of neurons + weights

¹ADAS: Advanced Driver Assistance System

Risk Minimization

- The goal is to find the minimizer of the risk functional

$$R(w) = \int \int p(x, y) L(y, f(x, w)) dx dy, \quad w \in \Omega$$

$p(x, y)$ is the joint distribution over data x , and target values y

i.e., find $f(x, w^*)$ that minimizes

$R(w)$ over the class of functions $f(x, w)$, $w \in \Omega$

where the joint PDF $p(x, y)$ is unknown and the only available information is contained in the training set, i.e.,

$$(x_i, y_i), \quad i = 1, \dots, N$$

Calculus of Variations

function

- $y(x)$ as an operator that for any input **value/vector** x , returns a value y .

e.g.: $y = mx + c$; $y=x^2$; $y=x^3$; $y=a^T x+b$

- Calculus used for function minimization over a space of parameter values

functional

- $F[p]$ as an operator that for any input function $p(x)$, returns a value F

e.g. Entropy

$$H[p] = - \int p(x) \ln p(x) dx$$

Risk $R(w)$

- Calculus of variations used for minimizing functionals over a space of functions

Empirical Risk Minimization Principle

$$R(w) = \int \int p(x, y) L(y, f(x, w)) dx dy, \quad w \in \Omega$$

- The risk functional is replaced by the *empirical risk functional*

$$R_{emp}(w) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i, w))$$

- Learning theory asks the following questions
 - What are the conditions for consistency?
 - How fast is the convergence rate?
 - How can one control generalization ability (on unseen examples from $F(x, y)$)?
 - How can one construct algorithms to control generalization ability?

Training:

Loss functions & Empirical Risk Minimization

- The learning problem is to choose from the given set of functions the one which best approximates the supervisor's response.
- To choose the best function, it makes sense to **minimize** a *loss* (or cost or discrepancy measure) between the **response of the supervisor and the learning machine**, given an input x

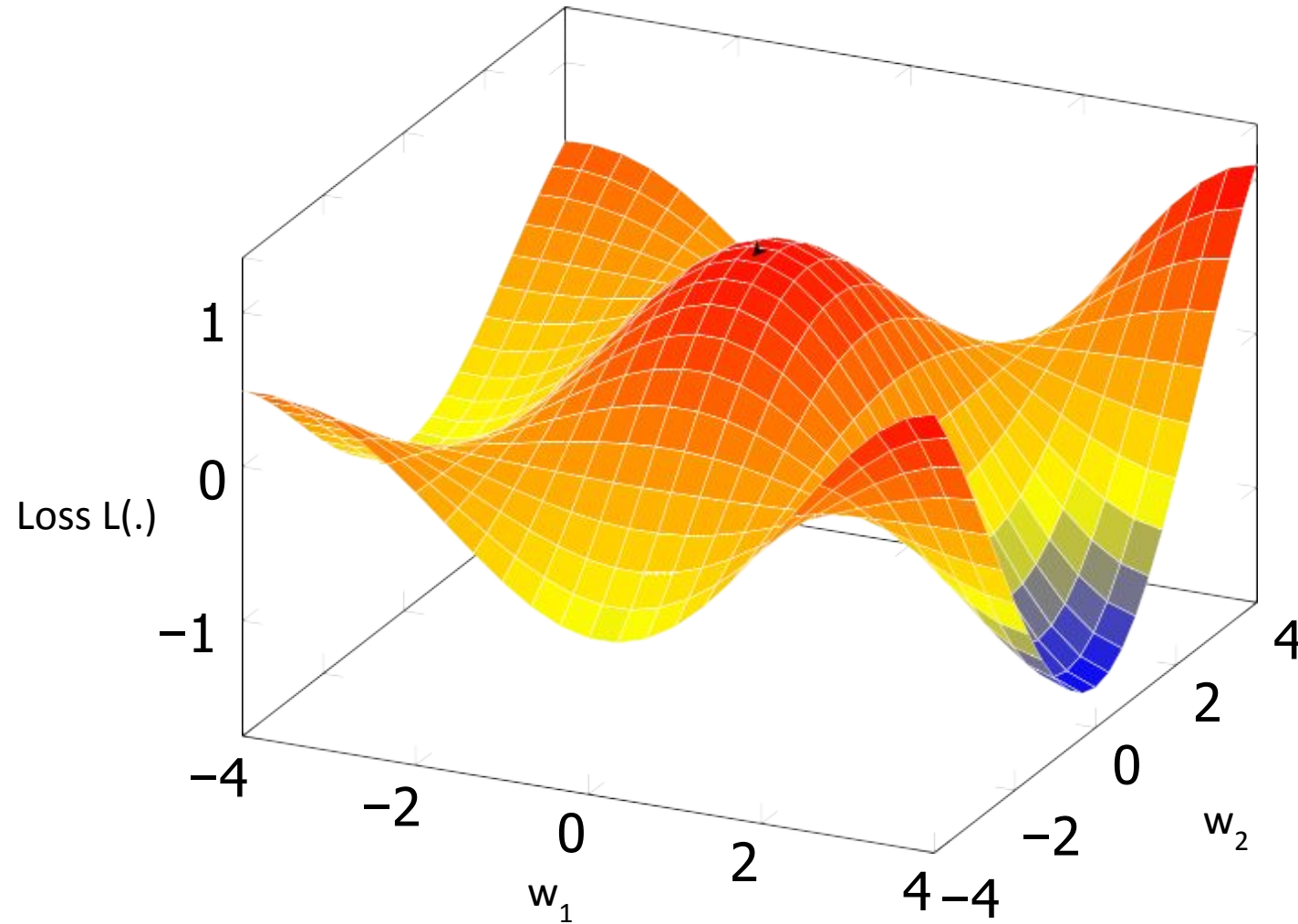
$$L(y, f(x, w)) \quad \begin{array}{ll} \text{Regression} & L(y, f(x, w)) = \|y - f(x, w)\|_2^2 \\ & L(y, f(x, w)) = \begin{cases} 0 & \text{if } y = f(x, w) \\ 1 & \text{if } y \neq f(x, w) \end{cases} \quad \text{Classification} \end{array}$$

- Minimizing the Empirical Risk over *all training samples*

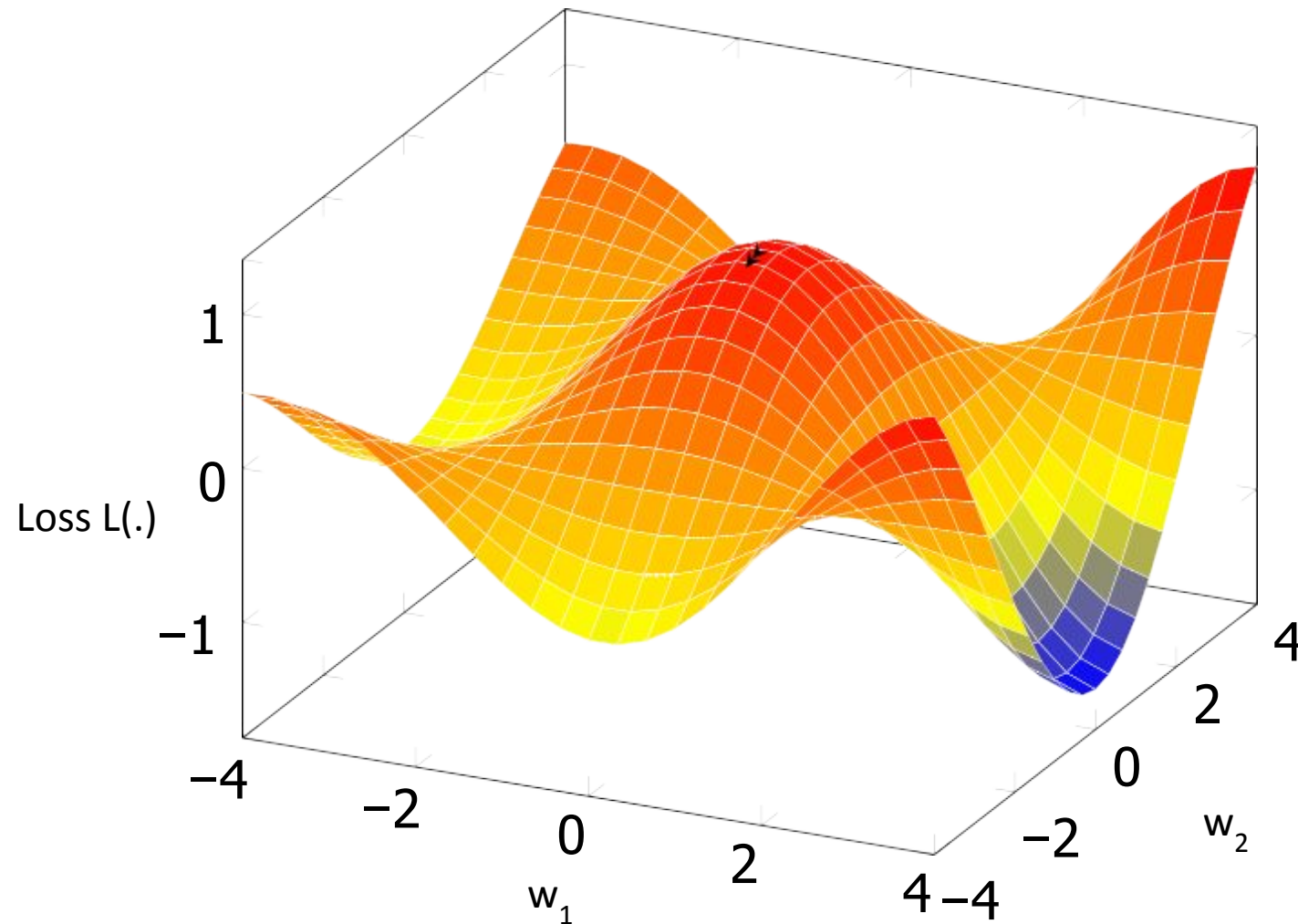
$$R_{emp}(w) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i, w))$$

Note: The empirical risk is an estimate of the *true* risk based on the samples that are accessible at training time

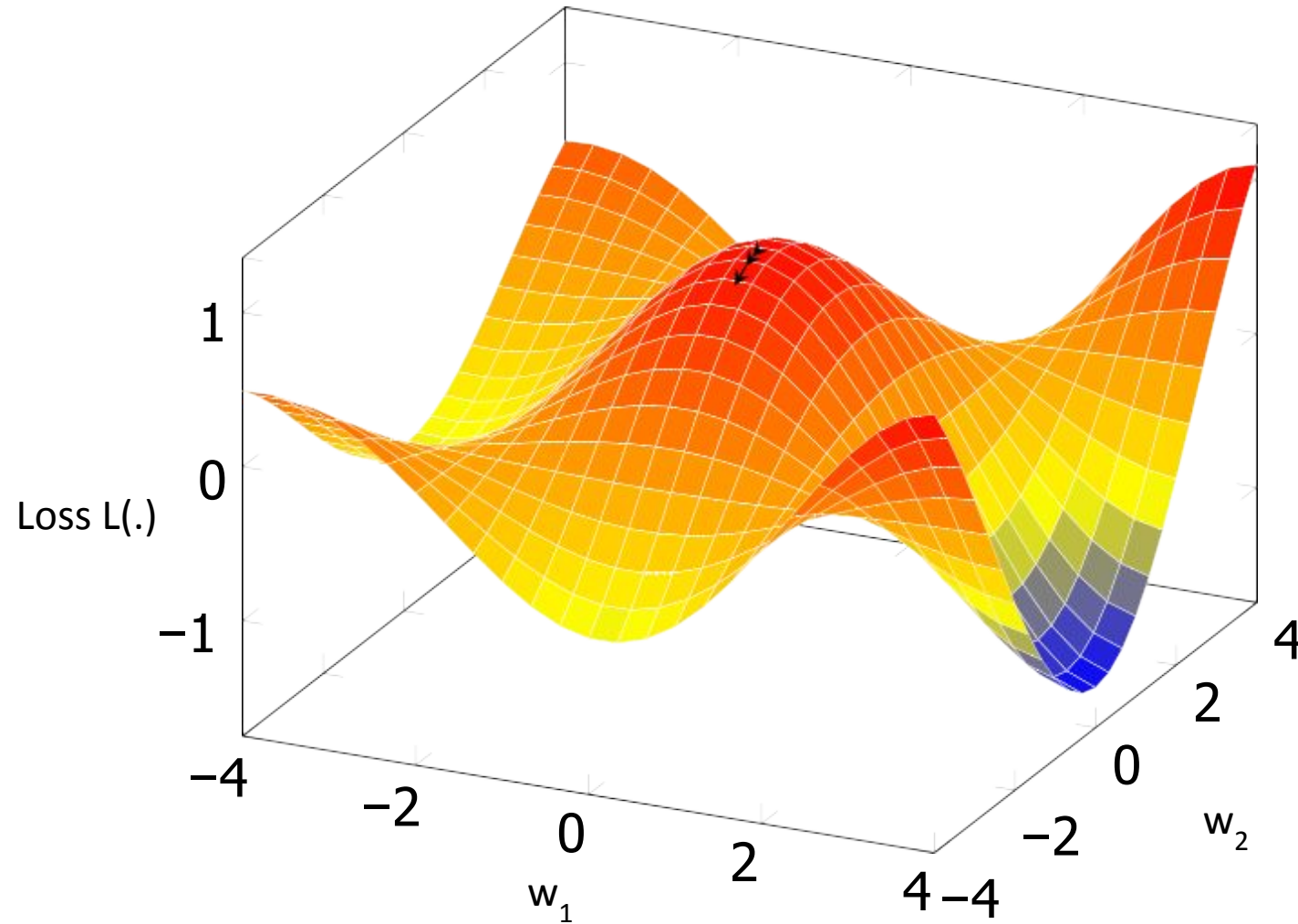
Gradient Descent for Loss Minimization



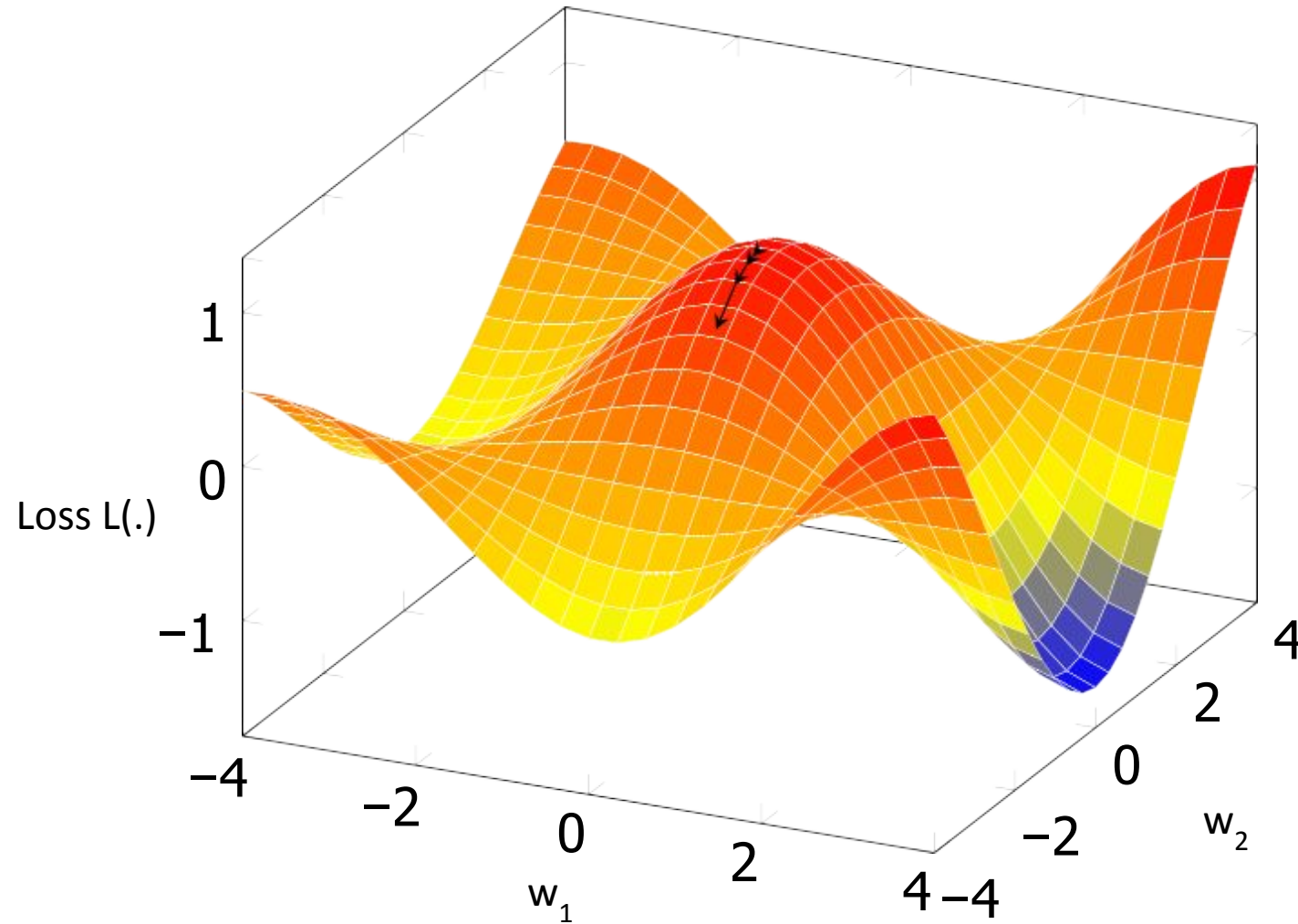
Gradient Descent for Loss Minimization



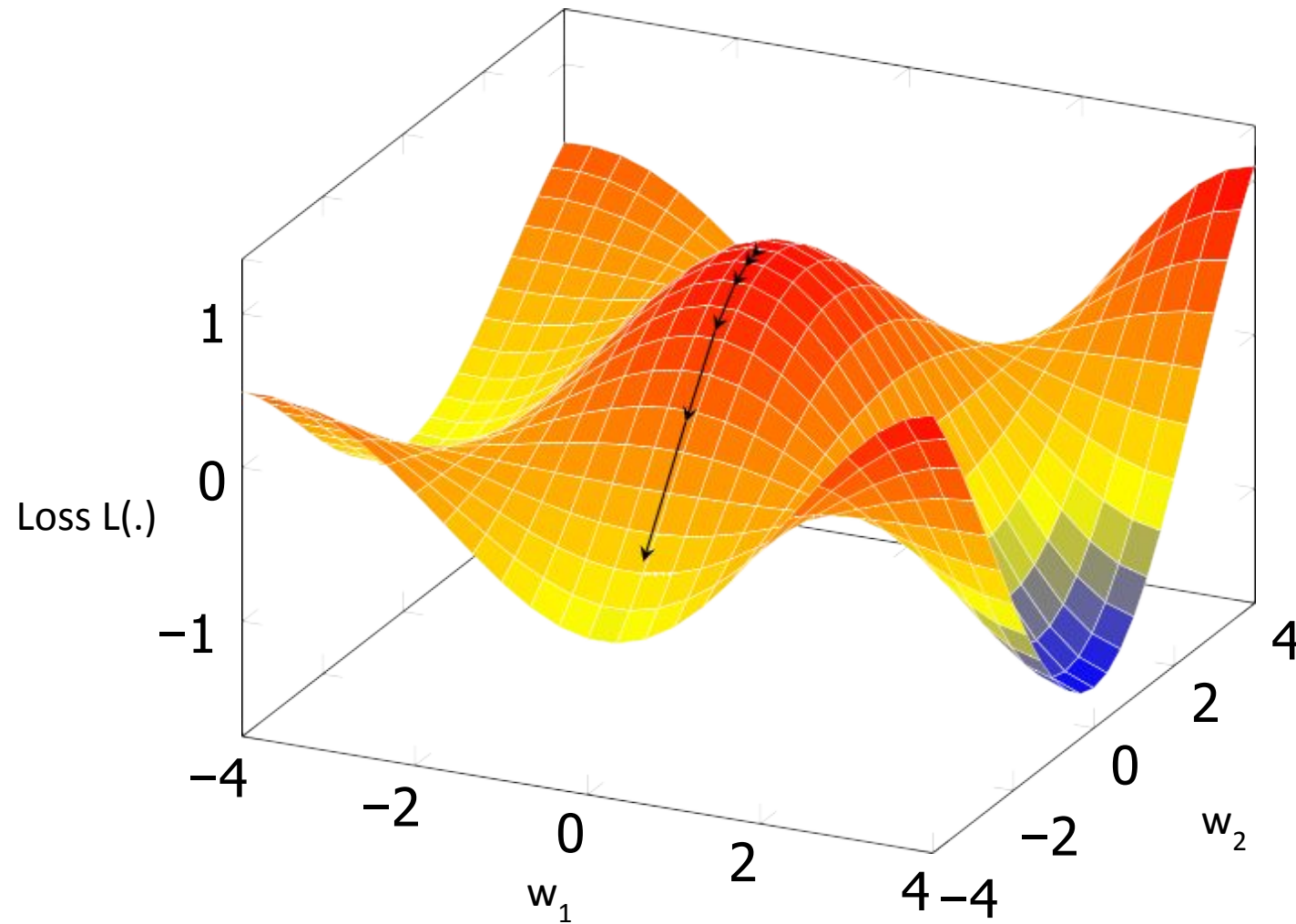
Gradient Descent for Loss Minimization



Gradient Descent for Loss Minimization



Gradient Descent for Loss Minimization



Performance Evaluation of Learning Tasks

- Measuring Performance: How well does a learned model work?
 - Performance is typically measured by estimating the TRUE ERROR RATE
 - UNSEEN DATA
- Evaluation Metrics (examples)
 - Classification: Accuracy
 - Regression: Mean Squared Error
 - Retrieval: Precision/Recall, F-Score
 - Ranking: mean Average Precision
 - Clustering: Normalized Mutual Information (NMI)

Next Class

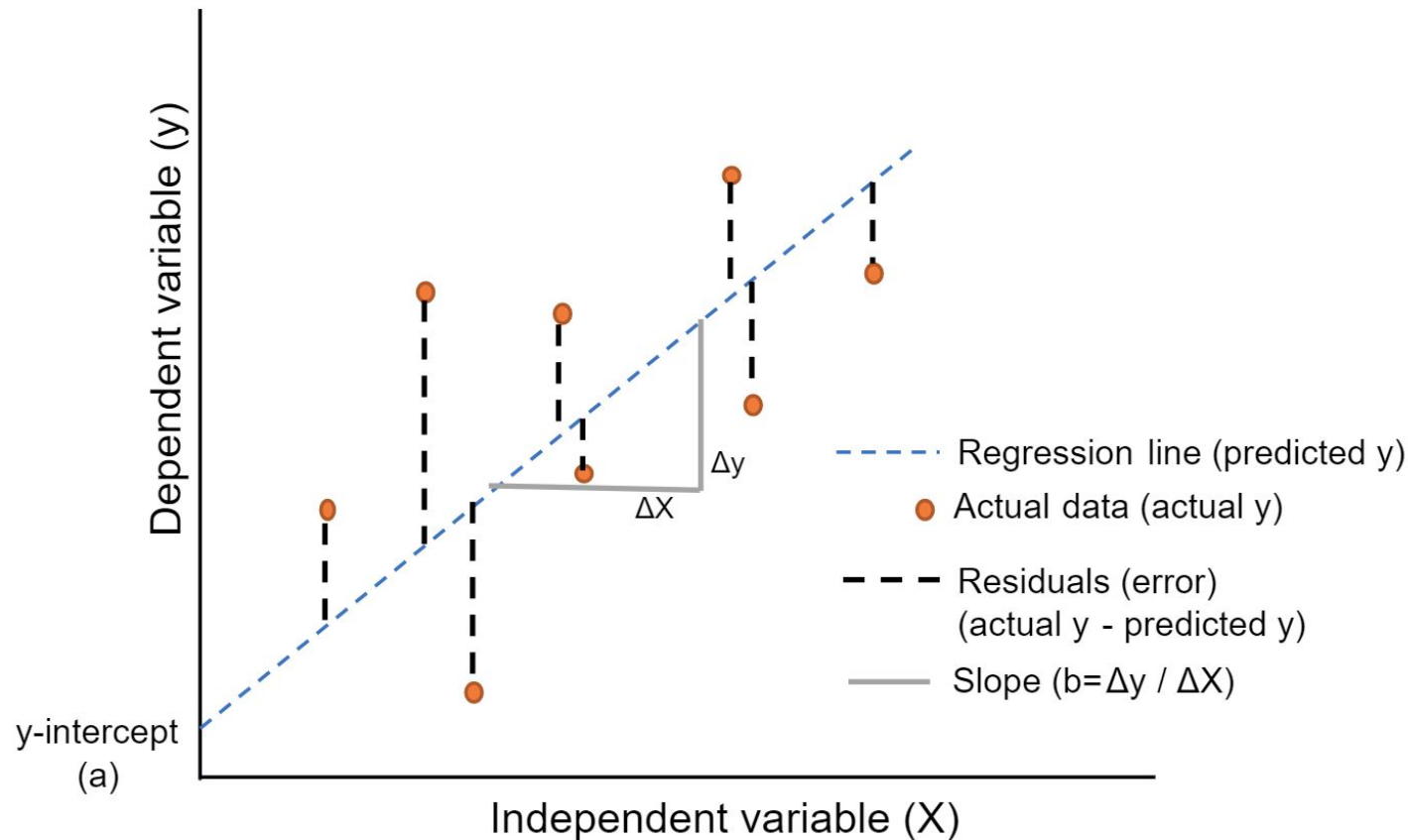
- Linear Models for Regression and Classification
- Formulation of Linear Regression
 - Closed-form Solution
 - Gradient Descent for Iterative Solution

Performance Evaluation of Learning Tasks

- Entire population is unavailable
- Finite set of training data, usually smaller than desired
- Naïve approach: use all available data
 - The final model will typically **overfit** the training data
 - More pronounced with high-capacity models (e.g., neural nets)
 - The true error rate is **underestimated**
 - Not uncommon to have 100% accuracy on training data

Linear Regression

- Predict real, continuous target variables $\hat{y} = f(x, w) = w * x + b$



Loss $L(y, f(x, w)) = \|y - f(x, w)\|_2^2$

$$\hat{y} = f(x, w, b) = w^\top x + b$$